Two Basic Operations on Stack :

- PUSH
- POP

Algorithms:

PUSH(Stack, N, Item)

This algorithm will insert Item at top to Stack having size N. Here initial index for stack is 0.

```
          if TOP >= N - 1 then
                    "Overflow"
          else
                    TOP = TOP + 1
                    Stack[TOP] = Item
```

POP(Stack, N)

This algorithm will delete the top element from the Stack having size N. Here initial index for stack is 0.

```
  if TOP = -1 then
          "Underflow"
  else
           Stack[TOP] = NULL
           TOP = TOP - 1
```

```
#include<stdio.h>
#include<process.h>
#include<stdlib.h>

#define MAX 5    //Maximum number of elements that can be stored

int top=-1,stack[MAX];
void push();
void pop();
void display();

void main()
{
   int ch;

   while(1)    //infinite loop, will end when choice will be 4
   {
      printf("\n*** Stack Menu ***");
      printf("\n\n1.Push\n2.Pop\n3.Display\n4.Exit");
      printf("\n\nEnter your choice(1-4):");
      scanf("%d",&ch);

      switch(ch)
      {
         case 1: push();
                 break;
         case 2: pop();
                 break;
         case 3: display();
                 break;
         case 4: exit(0);

         default: printf("\nWrong Choice!!");
      }
   }
}

void push()
{
   int val;

   if(top==MAX-1)
   {
      printf("\nStack is full!!");
   }
   else
   {
      printf("\nEnter element to push:");
```

```
        scanf("%d",&val);
        top=top+1;
        stack[top]=val;
    }
}

void pop()
{
    if(top==-1)
    {
        printf("\nStack is empty!!");
    }
    else
    {
        printf("\nDeleted element is %d",stack[top]);
        top=top-1;
    }
}

void display()
{
    int i;

    if(top==-1)
    {
        printf("\nStack is empty!!");
    }
    else
    {
        printf("\nStack is...\n");
        for(i=top;i>=0;--i)
            printf("%d\n",stack[i]);
    }
}
```

//

Rules

The mission is to move all the disks to some another tower without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi are –

- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.

- No large disk can sit over a small disk.

To write an algorithm for Tower of Hanoi, first we need to learn how to solve this problem with lesser amount of disks, say → 1 or 2. We mark three towers with name, **source**, **destination**and **aux** (only to help moving the disks). If we have only one disk, then it can easily be moved from source to destination peg.

If we have 2 disks –

- First, we move the smaller (top) disk to aux peg.
- Then, we move the larger (bottom) disk to destination peg.
- And finally, we move the smaller disk from aux to destination peg.

So now, we are in a position to design an algorithm for Tower of Hanoi with more than two disks. We divide the stack of disks in two parts. The largest disk (n^{th} disk) is in one part and all other (n-1) disks are in the second part.

Our ultimate aim is to move disk **n^{th}** from source to destination and then put all other (n-1) disks onto it. We can imagine to apply the same in a recursive way for all given set of disks.

The steps to follow are –

```
Step 1 – Move n-1 disks from source to aux
Step 2 – Move nth disk from source to dest
Step 3 – Move n-1 disks from aux to dest
```

A recursive algorithm for Tower of Hanoi can be driven as follows –

```
START

Procedure Hanoi(disk, source, dest, aux)



   IF disk < 2, THEN

      move disk from source to dest
```

```
   ELSE
      Hanoi(disc-1,beg,end,aux);             // Step 1
      move disk from source to dest          // Step 2
      Hanoi(disc-1,aux,beg,end);            // Step 3
   END IF

END Procedure
STOP
```

```
#include<iostream>
using namespace std;
int TOWER(int disc,char beg,char aux,char end)
{
   if(disc<2)
   {
      cout<<"  "<<beg<<"-->"<<end;
      return 0;
   }
   TOWER(disc-1,beg,end,aux);
   cout<<"  "<<beg<<"-->"<<end;
   cout<<"\n\n******Second function is called:";
   TOWER(disc-1,aux,beg,end);

}
int main()
{
   int n;
   char a='A',b='B',c='C';
```

```
  while(1)
  {
    cout<<"\n**Enter your disc number: ";
    cin>>n;
    if(n<=0)
        break;
    TOWER(n,a,b,c);
  }

}
```

////////////

	current symbol	operator stack	postfix string
1	A		A
2	*	*	A
3	B	*	A B
4	+	+	A B * {pop and print the '*' before pushing the '+'}
5	C	+	A B * C
6			A B * C +

The rule used in lines 1, 3 and 5 is to print an operand when it is read. The rule for line 2 is to push an operator onto the stack if it is empty. The rule for line 4 is if the operator on the top of the stack has higher precedence than the one being read, pop and print the one on top and then push the new operator on. The rule for line 6 is that when the end of the expression has been reached, pop the operators on the stack one at a time and print them.

2. A + B * C becomes A B C * +

Here the order of the operators must be reversed. The stack is suitable for this, since operators will be popped off in the reverse order from that in which they were pushed.

	current symbol	operator stack	postfix string
1	A		A
2	+	+	A
3	B	+	A B
4	*	+ *	A B
5	C	+ *	A B C
6			A B C * +

In line 4, the '*' sign is pushed onto the stack because it has higher precedence than the '+' sign which is already there. Then when the are both popped off in lines 6 and 7, their order will be reversed.

3. A * (B + C) becomes A B C + *

A subexpression in parentheses must be done before the rest of the expression.

	current symbol	operator stack	postfix string
1	A		A
2	*	*	A
3	(	* (	A B
4	B	* (	A B
5	+	* (+	A B
6	C	* (+	A B C
7	)	*	A B C +
8			A B C + *

Since expressions in parentheses must be done first, everything on the stack is saved and the left parenthesis is pushed to provide a marker. When the next operator is read, the stack is treated as though it were empty and the new operator (here the '+' sign) is pushed on. Then when the right parenthesis is read, the stack is popped until the corresponding left parenthesis is found. Since postfix expressions have no parentheses, the parentheses are not printed.

4. A - B + C becomes A B - C +

When operators have the same precedence, we must consider association. Left to right association means that the operator on the stack must be done first, while right to left association means the reverse.

current symbol	operator stack	postfix string

1	A		A
2	-	-	A
3	B	-	A B
4	+	+	A B -
5	C	+	A B - C
6			A B - C +

In line 4, the '-' will be popped and printed before the '+' is pushed onto the stack. Both operators have the same precedence level, so left to right association tells us to do the first one found before the second.

5. A * B ^ C + D becomes A B C ^ * D +

Here both the exponentiation and the multiplication must be done before the addition.

	current symbol	operator stack	postfix string
1	A		A
2	*	*	A
3	B	*	A B
4	^	* ^	A B
5	C	* ^	A B C
6	+	+	A B C ^ *
7	D	+	A B C ^ * D
8			A B C ^ * D +

When the '+' is encountered in line 6, it is first compared to the '^' on top of the stack. Since it has lower precedence, the '^' is popped and printed. But instead of pushing the '+' sign onto the stack now, we must compare it with the new top of the stack, the '*'. Since the operator also has higher precedence than the '+', it also must be popped and printed. Now the stack is empty, so the '+' can be pushed onto the stack.

6. A * (B + C * D) + E becomes A B C D * + * E +

	current symbol	operator stack	postfix string
1	A		A
2	*	*	A
3	(	* (	A
4	B	* (	A B
5	+	* (+	A B
6	C	* (+	A B C
7	*	* (+ *	A B C
8	D	* (+ *	A B C D
9	)	*	A B C D * +
10	+	+	A B C D * + *
11	E	+	A B C D * + * E
12			A B C D * + * E +

A summary of the rules follows:

1. Print operands as they arrive.

2. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.

3. If the incoming symbol is a left parenthesis, push it on the stack.

4. If the incoming symbol is a right parenthesis, pop the stack and print the operators until you see a left parenthesis. Discard the pair of parentheses.

5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.

6. If the incoming symbol has equal precedence with the top of the stack, use association. If the association is left to right, pop and print the top of the stack and

then push the incoming operator. If the association is right to left, push the incoming operator.

7. If the incoming symbol has lower precedence than the symbol on the top of the stack, pop the stack and print the top operator. Then test the incoming operator against the new top of stack.

8. At the end of the expression, pop and print all operators on the stack. (No parentheses should remain.)